Thread Exercise solution
Q#6

```
1  public class ThreadInfo {
2    public static void main(String[] args){
3      Thread currentThread = Thread.currentThread();
4      System.out.println("Thread Name : " + currentThread.getName());
5      System.out.println("Thread ID : " + currentThread.getId());
6      System.out.println("Thread Priority : " + currentThread.getPriority());
7      System.out.println("Thread State : " + currentThread.getState());
8      System.out.println("Thread is Daemon : " + currentThread.isDaemon());
9      System.out.println("Thread is Alive : " + currentThread.isAlive());
10     System.out.println("Thread is Interrupted : " + currentThread.isInterrupted());
11   }
12 }
```

Q#7

```
1  class SimpleThread extends Thread {
2    public SimpleThread(String str) { //take the name of a thread as argument
3      super(str);   //call super class constructor to set the name of the thread
4    }
5    public void run() {
6      for (int i = 0; i < 10; i++) { //print thread name and its priority 10 times
7        System.out.println(i+ " "+getName()+ " Priority = " + getPriority());
8      }
9      System.out.println("Done! " + getName());
10   }
11 }
12 public class ThreadPriority {
13   public static void main(String[] args) {
14     Thread t1 = new SimpleThread("Boston"); //create a thread named 'Boston'
15     t1.start();  //start the thread
16     t1.setPriority(10); //set its priority to 10
17     Thread t2 = new SimpleThread("New York");
18     t2.start();
19     t2.setPriority(5);
20     Thread t3 = new SimpleThread("Seoul");
21     t3.start();
22     t3.setPriority(1);
23   } }
```

Q#8

```
class Printer{
   boolean evenTurn = false;//initially it is odd writer's turn
   synchronized void evenPrinter(int n){
      try{
         while(!evenTurn)//while it is odd writer's turn, just wait
            wait();
      }catch(InterruptedException e){}
      System.out.println("Even Number from evenThread: " + n);
      evenTurn = false; //even writer has done, now it is odd writer's turn
       notify();//notify odd writer
   }
   synchronized void oddPrinter(int n){
      try{
         while(evenTurn)//while it is even writer's turn, just wait
            wait();
      }catch(InterruptedException e){}
      System.out.println("Odd Number from OddThread: " + n);
      evenTurn = true;//odd writer has done, now it is even writer's turn
      notify();//notify even writer
   }
}
class PrinterThrd implements Runnable{
   int initial; //1 or 2, used to identify odd writer and even writer threads
   final int MAX = 20;
   Printer prnt;
   PrinterThrd(Printer p, int n){
      initial = n;
      prnt = p;
   }
   public void run(){
      if(initial%2 == 1)//if initial is odd, call oddPrinter() method of Printer class
         for(int i = initial; i <=MAX; i+=2)
            prnt.oddPrinter(i);
      else //if initial is even, call evenPrinter method of Printer class
         for(int i = initial; i <=MAX; i+=2)
            prnt.evenPrinter(i);
   }
}
public class NumberPrinter {
   public static void main(String args[]) {
      Printer p = new Printer();
      PrinterThrd evenPrnt = new PrinterThrd(p, 2);//even numbers printer
      PrinterThrd oddPrnt = new PrinterThrd(p, 1);//odd numbers printer
      Thread evenThrd = new Thread(evenPrnt);
      Thread oddThrd = new Thread(oddPrnt);
      evenThrd.start();
      oddThrd.start();
   }
}
```

Q#9

A) No, because the printString() method of StringPrinter class is non-synchronized and can be accessed by the three threads simultaneously.

B) make the printString() method of StringPrinter class a synchronized method, so that it will be accessed by only one thread at a time. The one that accessed this method prints a complete sentence before other threads start printing the other sentences.

Q#10

Yes, here a synchronized block is used. The portion of the run method of the threads that calls the printString() method is synchronized. That means only one thread can call the method at a time.

Q#12

A) No. to correct it rewrite the MyBuffer class as follows:

```
class MyBuffer{
   int[] arr = new int[4]; //buffer size is 4
   int readIndex = 0; //read index
   int writeIndex = 0; //write index
   int count = 0; //number of products in the buffer
   synchronized void produce(int value)throws Exception{
      while(count == 4) //while number of products in the buffer is 4, i.e while buffer is full
         wait();              //just wait
      arr[writeIndex] = value;  //store a product at writeIndex
      System.out.println("producer writes: " + value + " at "+writeIndex);//print a message
      writeIndex = ++writeIndex%4;//increament write index, go back to 0 if it is 4.
      Count++; //increament the number of products in the buffer
      Notify(); //notify the consumer thread
   }
   synchronized void consume() throws Exception{
      while(count == 0)  //while number of products in the buffer is 0, i.e while buffer is empty
         wait();              //wait
      System.out.println("consumer reads: " + arr[readIndex] + " from "+readIndex);//consume the
                                                  //product stored at read index and print a message
      readIndex = ++readIndex%4; //increament read index, go back to 0 if it is 4
      Count--; //decreament the number of products available in the buffer
      Notify();  //notify the producer thread
   }
}
```

B)

```
class MyBuffer{
   ArrayBlockingQueue<Integer> abq;
   MyBuffer(){
      abq = new ArrayBlockingQueue<>(4);
   }
   void produce(int n)throws InterruptedException{
      abq.put(n);
      System.out.println("producer writes: " + n);
   }
   void consume()throws InterruptedException{
      int value = abq.take();
      System.out.println("consumer reads: " + value);
   } }
```